Q1. Evaluate two AI answers about a data file — 3 points

What to study:

- Reading CSV files using `readr::read_csv()`.
- Inspecting datasets.
- Identifying the unit of analysis.
- Recognizing missing values and data types.
- Checking whether AI descriptions agree with the actual data.

Know these commands

```
data = readr::read_csv("file.csv")

# See the dataset's structure
dplyr::glimpse(data)

# Number of rows and columns
nrow(data)
ncol(data)
dim(data)

# Column names
names(data)

# First six rows
head(data)

# Summary of the columns
summary(data)
```

What you should be able to do

Suppose an AI says:

> This dataset contains 500 observations representing individual customers.

You should know how to check that claim.

```
nrow(data)
```

If it returns 450, the AI's claim about the number of observations is incorrect.

But if it returns 500, that only confirms the row count. You still need to establish whether each row represents a customer.

Remember: The unit of analysis is what one observation or row represents.

Examples:

- One row per university → institution.
- One row per Airbnb listing → listing.
- One row per customer → customer.

Don't accept an AI's description without checking the actual dataset.

Q2. Git and GitHub Desktop — 1 point

This question is based on your portfolio assignment.

Review the differences between Git and GitHub.

Term	Meaning
Git	A version control system that tracks changes to files.
GitHub	An online platform for hosting and collaborating on Git repositories.
Repository	A project's files and their version history.
Commit	A saved snapshot of changes.
Push	Sends local commits to a remote repository.
Pull	Retrieves and integrates changes from a remote repository.
Clone	Creates a local copy of a remote repository.
Pull request	Proposes changes for review and possible integration.

Example

You change an R script in RStudio.

1. You save the file.
2. Git recognizes that the file has changed.
3. You stage the change.

4. You commit the change.
5. You push the commit to GitHub.

Important distinction: Saving a file does not automatically mean you've committed and pushed the change to GitHub.

Q3. Predict what R code prints — 2 points

This is a high-priority topic because AI can easily make mistakes when predicting R output.

You need to understand indexing, lists, vectors, and data frames.

Understand `[ ]` versus `[[ ]]`

Suppose:

```
x = list("apple", "banana", "orange")
```

Using single brackets:

```
x[2]
```

Returns a list containing the second element.

Using double brackets:

```
x[[2]]
```

Returns the second element itself: `"banana"`.

Think of it like this:

- `[2]` → give me a smaller list containing item 2.
- `[[2]]` → give me item 2.

Data frame indexing

Suppose:

```
df = data.frame(
  name = c("Alex", "Sam", "Jordan"),
  score = c(80, 90, 85)
```

)

Code	What it returns
`df[1, ]`	First row, all columns
`df[, 2]`	Second column
`df[2, 1]`	Row 2, column 1
`df$name`	The `name` column
`nrow(df)`	Number of rows
`ncol(df)`	Number of columns
`dim(df)`	Rows and columns

Also review the pipe

x |> head()

This passes `x` into `head()`.

Conceptually, it is similar to:

head(x)

Your practice strategy

For any AI prediction, run the code yourself.

```
# First, inspect the object
str(x)

# Then run the exact expression
x[2]
x[[2]]
```

Do not assume an AI understands R's indexing rules correctly.

Q4. Reading a nested JSON file — 3 points

Study this carefully. Your professor explicitly says you must identify two AI errors and correct the code.

Review:

- `jsonlite::fromJSON()`
- Lists and nested objects.
- `$` for accessing named components.
- `[[ ]]` for extracting a component.
- `str()` for inspecting structure.
- `names()` for identifying available components.

Basic example

Imagine a JSON response with this structure:

```
{
  "name": "Alex",
  "location": {
    "city": "Oxford",
    "state": "Ohio"
  }
}
```

Read it into R:

x = jsonlite::fromJSON("data.json")

Inspect it:

str(x)

Access the name:

x$name

Access the nested city:

x$location$city

The expected city is `"Oxford"`.

The most important concept

Inspect the structure before guessing how to extract data.

If an AI gives you:

x$location$city

but `location` is actually a list of several records, the correct extraction may be different.

Check:

str(x)
names(x)

Then determine whether you have:

- A list.
- A data frame.
- A list containing data frames.
- Nested lists.
- A vector.

The correct extraction depends on the actual structure.

I cannot identify the two specific AI errors your professor will use from the announcement alone. Practice identifying structural mismatches rather than memorizing two guessed errors.

Q5. Multiple choice about a technology used in class — 1 point

The announcement doesn't identify the technology, so we shouldn't guess which one will appear.

Review the technologies you've used in ISA 401, including:

- Git and GitHub.
- Web scraping with `rvest`.
- APIs.
- JSON.

- R packages.
- Structured text extraction.

For each technology, be able to explain:

1. What it does.
2. When you would use it.
3. How you used it in class.

Pay special attention to the distinction between scraping a webpage and requesting data from an API.

Q6. API requests and documentation — 4 points

This is another major priority because it is worth four points.

Your professor says you'll evaluate two AI-written requests, read the API documentation, and run code.

You've previously worked with APIs including Weather.gov and IP geolocation.

Understand the difference between an API URL and a webpage URL

A normal webpage might be:

https://example.com/weather

An API endpoint might return structured data:

https://api.example.com/weather?city=Oxford

The second example is illustrative, not a real endpoint.

Know how to inspect a response

A typical API workflow might look like this:

```
url = "YOUR_API_ENDPOINT"

response = httr2::request(url) |>
  httr2::req_perform()

data = response |>
  httr2::resp_body_json()
```

str(data)

The endpoint and any required parameters must come from the actual API documentation.

When checking an AI-generated request, ask:

1. Is the endpoint correct?

Does it match the API documentation?

2. Are the parameters correct?

Are they spelled correctly? Are required parameters missing?

3. Is the requested data actually available?

An API may return a different structure than the AI expects.

4. Is the extraction code correct?

For example:

data$properties$gridX

only works if those components exist in the returned object.

5. Does the actual output match the AI's explanation?

Run the code and inspect the result.

Your Weather.gov example

You previously worked with:

https://api.weather.gov/points/26.142,-81.7948

The response contains a `properties` object, including `gridX` and `gridY`.

The important skill is understanding how to use the API response's actual structure rather than guessing.

Exam-day rule: Trust the API documentation and the actual response over the AI's explanation.

Q7. Web scraping across multiple pages — 4 points

This is the topic I would spend the most hands-on practice time on with you.

It's worth four points, and your recent confusion about SelectorGadget, CSS selectors, and URLs is directly relevant.

You need to understand the entire scraping workflow.

A. The basic scraping pattern

```
url = "https://example.com"

page = rvest::read_html(url)

results = page |>
  rvest::html_elements("YOUR_CSS_SELECTOR") |>
  rvest::html_text2()
```

The example URL and selector above are placeholders.

Understand each function:

Function	Purpose
`read_html()`	Reads the webpage's HTML.
`html_elements()`	Finds HTML elements matching a CSS selector.
`html_text2()`	Extracts readable text from the elements.
`html_attr("href")`	Extracts a link's URL.
`url_absolute()`	Converts relative URLs to absolute URLs.

B. Know how CSS selectors work

This is especially important based on our conversation.

Selector	Meaning
`"a"`	All `<a>` elements.
`"td"`	All `<td>` table cells.
`".my-class"`	Elements with class `my-class`.
`"#my-id"`	The element with ID `my-id`.
`"a.my-class"`	`<a>` elements with class `my-class`.
`"div a"`	`<a>` elements anywhere inside a `<div>`.
`"div > a"`	`<a>` elements that are direct children of a `<div>`.

Your Miami policy-page example

You have this code:

```
"https://miamioh.edu/policy-library/students/undergraduate/academic-regulations/" |>
  rvest::read_html() |>
  rvest::html_elements("a.news-lockup__body-link") |>
  rvest::html_attr("href") |>
  rvest::url_absolute("https://miamioh.edu/policy-library/") ->
  mupolicy

mupolicy
```

Here's what each part does:

Step 1: Read the page.

rvest::read_html()

Gets the webpage's HTML.

Step 2: Find the policy links.

rvest::html_elements("a.news-lockup__body-link")

Finds `<a>` elements with the class `news-lockup__body-link`.

Notice the **two underscores** between `lockup` and `body`.

Step 3: Extract the links.

rvest::html_attr("href")

Extracts each link's `href` attribute.

Step 4: Convert the links to absolute URLs.

rvest::url_absolute("https://miamioh.edu/policy-library/")

If the HTML contains:

refunds.html

the resulting URL would be:

https://miamioh.edu/policy-library/refunds.html

That is an illustration of relative URL conversion; the correct base depends on the webpage's URL structure.

Step 5: Save the results.

-> mupolicy

Assigns the result to `mupolicy`.

Important correction about `url_absolute()`

For relative URLs on your policy page, the safest approach is to use the **actual page URL as the base**:

```
"https://miamioh.edu/policy-library/students/undergraduate/academic-regulations/" |>
  rvest::read_html() |>
  rvest::html_elements("a.news-lockup__body-link") |>
  rvest::html_attr("href") |>

rvest::url_absolute("https://miamioh.edu/policy-library/students/undergraduate/academic-regulations/") ->
  mupolicy
```

Why? Because a relative link like `refunds.html` normally resolves relative to the directory containing the current page. However, if your course materials deliberately use a different base

because of the site's link structure, follow the assignment's expected approach and verify the resulting URLs.

Do not assume every website's relative links work the same way.

C. SelectorGadget: What you must actually understand

SelectorGadget helps you identify CSS selectors, but you must check what they select.

For example:

#card--69-body > a

means:

- `#card--69-body`: the element with that ID.
- `>`: its direct child.
- `a`: a link.

Meanwhile:

a.news-lockup__body-link

means:

- `a`: a link.
- `.news-lockup__body-link`: with that class.

Both can identify the same link on a page, but the class-based selector may be more appropriate for finding links across multiple cards.

Always test the selector against the collection you want to scrape.

For the FSB Directory, for example, `td` selects table cells broadly. It doesn't automatically isolate people's names or job titles.

D. Scraping across multiple pages

Your professor specifically says Q7 spans several pages.

Review the technique your class used for:

- Constructing multiple URLs.
- Reading each webpage.
- Extracting the same type of information from each page.

- Combining results into one data frame.

A common pattern is to write a function that scrapes one page and then apply it to multiple pages.

Illustrative structure:

```
scrape_page = function(url) {
  rvest::read_html(url) |>
    rvest::html_elements("YOUR_CSS_SELECTOR") |>
    rvest::html_text2()
}
```

Then:

```
results = purrr::map(urls, scrape_page)
```

This returns a list of results, one for each URL. If you need a single combined vector or data frame, the next step depends on what each page returns.

What to practice

Make sure you can explain:

- How to identify the right selector.
- How to extract text versus attributes.
- How to convert relative URLs.
- How to handle multiple pages.
- How to check whether the returned results make sense.

Q8. Tidy data — 1 point

Review the three principles of tidy data:

1. Each variable forms a column.
2. Each observation forms a row.
3. Each value occupies a cell.

Example of untidy data

Institution	2024	2025
University A	100	120

University B	200	220

Here, the year is encoded in the column names.

A tidy version would be:

Institution	Year	Value
University A	2024	100
University A	2025	120
University B	2024	200
University B	2025	220

Now each variable has its own column, and each row represents one institution-year observation.

Review `tidyr::pivot_longer()` and `tidyr::pivot_wider()` if they were covered in class.

- `pivot_longer()` turns multiple columns into rows.
- `pivot_wider()` turns values into columns.

Q9. Technically correct data — 1 point

This is related to the same file as Q8, but it tests a different concept.

A dataset can be tidy and still have technically incorrect data types.

For example:

id	hbcu_indicator	carnegie_category
1	TRUE	Research
2	FALSE	Master's

An institution ID, an HBCU indicator, and a Carnegie category represent different kinds of information.

An ID may be stored as a number even though arithmetic on the ID makes no sense.

A categorical variable may need to be represented as a factor.

For example:

data$id = as.factor(data$id)

data$hbcu_indicator = as.factor(data$hbcu_indicator)

data$carnegie_category = as.factor(data$carnegie_category)

Whether those exact conversions are appropriate depends on the definitions and requirements of the dataset.

Remember: Tidy data concerns the arrangement of observations and variables. Technical correctness concerns whether the data's representation and types make sense.

Q10. dplyr — 1 point

Review `mutate()`, `summarise()`, and `group_by()`.

`mutate()`

Adds or changes columns while generally preserving the number of rows.

```
data |>
  dplyr::mutate(double_value = value * 2)
```

`summarise()`

Calculates summary values and generally reduces the number of rows.

```
data |>
  dplyr::summarise(average = mean(value, na.rm = TRUE))
```

`group_by()`

Changes how subsequent dplyr operations work by applying them separately to groups.

```
data |>
  dplyr::group_by(category) |>
```

dplyr::summarise(average = mean(value, na.rm = TRUE))

This produces one summary row per category, assuming the usual grouping behavior and no additional grouping complications.

Exam trap: `group_by()` doesn't itself calculate summaries or remove rows. It establishes the groups used by later operations.

Q11. Structured text extraction — 2 points

This question asks you to run AI-written code in your class project and fix it.

Based on the material you've worked on, review string extraction and regular expressions.

Useful functions include:

stringr::str_detect()
stringr::str_extract()
stringr::str_remove()
stringr::str_replace()
stringr::str_replace_all()

Example: Remove a prefix

Suppose:

location = "Hometown Charlotte, North Carolina"

To remove `"Hometown "`:

stringr::str_remove(location, "^Hometown ")

The result is:

Charlotte, North Carolina

The `^` means the beginning of the string.

Example: Detect text

stringr::str_detect(location, "Hometown")

Returns `TRUE` if the string contains the specified text and `FALSE` otherwise.

Example: Extract a pattern

stringr::str_extract(text, "[0-9]+")

Extracts the first sequence of one or more digits.

How to debug AI-generated extraction code

1. Run the exact code.
2. Read the error message.
3. Inspect the input with `head()` or `str()`.
4. Test the regular expression on a few examples.
5. Check whether the output is missing text or extracting too much.
6. Correct the code and test it again.

Don't just look at the code and decide it looks right. **Check the actual extracted output.**

Q12. Short answer about a topic discussed in class — 1 point

The announcement doesn't specify which topic this is, so I wouldn't spend time guessing.

Review your lecture notes for concepts discussed recently, particularly:

- Data analysis and visualization.
- APIs and web scraping.
- R versus SQL versus spreadsheets.
- Data types and data quality.
- The role of programming in reproducible analysis.

Prepare a concise definition and one example for each major concept.

Q13. True/False about an R package — 1 point

Review the packages used in class and what each does.

Package	Primary purpose
`readr`	Reading rectangular text data such as CSV files.

`dplyr`	Manipulating and summarizing data frames.
`rvest`	Scraping HTML webpages.
`jsonlite`	Reading and converting JSON data.
`httr2`	Making HTTP requests, including API requests.
`purrr`	Applying functions to elements of lists or vectors.
`stringr`	Working with and extracting text.
`tidyr`	Reshaping data into tidy formats.
`DBI`	Providing an interface for database operations in R.

Be able to identify which package would be appropriate for a task.

For example:

- Scrape a webpage → `rvest`.
- Parse JSON → `jsonlite`.
- Read a CSV → `readr`.
- Apply a function to each URL → `purrr`.
- Remove a text prefix → `stringr`.

---

2. How I Would Prioritize Your Studying

You have 25 points total. Q6 and Q7 alone account for 8 points, or **32% of the exam**.

I'd organize your study time like this:

Priority	Questions	Points	What to do
1	Q7: Web scraping	4	Write and debug scraping code yourself.

2	Q6: API requests	4	Read API documentation and test requests.
3	Q1 and Q4: Data and JSON	6	Inspect actual data structures and verify AI answers.
4	Q3 and Q11: R output and text extraction	4	Run code, predict output, and fix errors.
5	Q2, Q5, Q8–Q10, Q12–Q13	7	Review concepts and practice concise explanations.

This is a starting point, not a guarantee of the exam's difficulty. You should also spend time on any topic where you cannot confidently explain the code.

3. The Most Important Skill: Checking AI Answers

Your professor's announcement gives you a very useful clue about the exam's design.

You are not being tested on whether you can get an AI to produce an answer. You are being tested on whether you can verify that answer.

Use this process for almost any AI-generated R answer:

Step 1: Identify the claim

What exactly is the AI saying?

For example:

> This code returns 10 rows and 5 columns.

Step 2: Run the code

Don't rely on the AI's explanation.

Step 3: Inspect the result

```
dim(data)
str(data)
head(data)
```

Use the command that best tests the claim.

Step 4: Compare the actual result with the claim

If the output contradicts the explanation, the explanation is wrong.

Step 5: Explain the correction

Write a short explanation that identifies the error and the correct behavior.

For example:

> The AI incorrectly predicted the output dimensions. Running `dim(data)` shows the actual number of rows and columns.

That is the kind of reasoning you should practice.

---

4. Your Final Exam Checklist

Before the exam, make sure you can do these things without needing to look up every step.

R fundamentals

- Explain `[ ]` versus `[[ ]]`.
- Predict the output of simple R expressions.
- Inspect an object's structure with `str()` and `glimpse()`.
- Identify the unit of analysis.
- Recognize incorrect data types.

JSON and APIs

- Read JSON using `jsonlite::fromJSON()`.
- Navigate nested lists.
- Read API documentation.
- Construct and run a request.
- Inspect the response and extract the correct field.
- Identify an AI's incorrect assumptions about an API response.

Web scraping

- Use `rvest::read_html()`.
- Understand CSS selectors.

- Use SelectorGadget to find candidate selectors.
- Extract text with `html_text2()`.
- Extract links with `html_attr("href")`.
- Understand relative versus absolute URLs.
- Scrape multiple pages and combine results.
- Troubleshoot selectors that return zero results.

Data manipulation and text

- Explain tidy data.
- Distinguish `mutate()` from `summarise()`.
- Explain how `group_by()` affects summaries.
- Use `stringr` functions to extract or clean text.
- Test whether code works instead of trusting an AI's answer.

Git and packages

- Distinguish Git from GitHub.
- Explain commit, push, pull, and clone.
- Identify the purpose of the major R packages used in class.

My recommendation for you specifically

Because you've been struggling with CSS selectors and how scraping code connects to the webpage, **don't just reread the scraping notes. Open RStudio and practice scraping the Miami policy page from scratch.**

Then try a second task where you must extract names or titles rather than URLs.

For each one, force yourself to explain:

1. What page am I reading?
2. What HTML element am I selecting?
3. Why is this the correct CSS selector?
4. Am I extracting text or an attribute?
5. What should the output look like?
6. How will I know whether it worked?

That same reasoning will help you evaluate AI answers during the exam.

One final point: Your professor says one website can only be opened by a human and scraped through R. The announcement doesn't identify the website, so I wouldn't assume it is the Miami

policy page or the FSB Directory. Review the scraping workflow you used in class so you can apply it to whichever site appears.

If you can independently run, inspect, and troubleshoot R code, you'll be much better prepared than if you simply memorize what each function does.